

Project - CSE 25

The project is your opportunity to apply the concepts learned throughout this course to tackle real-world problems within a team environment. You will work **in groups of three students**. Each group will be assigned a TA or tutor who will serve as a mentor throughout the process.

Your project should reflect the core themes of this course: artificial intelligence and machine learning. As this is an introductory course, we advise you to be ambitious but realistic. Large-scale networks can take days to train, so you should consider scaling back your problem size if you face time or compute constraints. Keep in mind that Datahub GPUs are shared across classes and may be busy in the days before the deadline.

We provide several example projects for your consideration. The code for the sample projects is available on Datahub: <https://datahub.ucsd.edu/hub/user-redirect/git-pull?repo=https://github.com/miaminnes/cse25-sp26&branch=main&urlpath=lab/tree/cse25-sp26/> However, you are welcome to pursue an original topic as long as it is relevant to the course and approved by your mentor.

Important: The goal of this project is to guide you through the complete machine learning lifecycle. Consequently, your grade will be based largely on your effort and the quality of your analysis rather than just the quantitative performance of your model. We are most interested in your ability to clearly explain your modeling choices, implement and analyze your algorithms correctly, and thoughtfully interpret your results and limitations. Strong projects demonstrate a deep engagement with the material and extensive analysis, even if their quantitative performance is modest.

Additional notes:

1. You will have at least one check-in meeting with your instructional staff mentor. Teams are also encouraged to reach out to their mentor with questions or for support at other points if needed.
2. All code must be written in Python. You may use external libraries or packages. If you do, you are responsible for understanding how they work. You may use open source code, as long as you provide proper attribution.
3. Your project report must be organized, clearly written, and neatly typed. We recommend using LaTeX and the NeurIPS format. The report should contain the sections listed in the "Project Structure" section below. Speak with your assigned instructional staff mentor if you have a good reason to deviate from this structure. Use a consistent style for your citations. Popular citation styles include APA, IEEE, ACM.
4. You will also submit code as part of your project. You can zip your file(s) and upload them to Gradescope and/or provide a Github link in the report. It's your responsibility to ensure that the files / links can be opened by the CSE 25 instructional team.
5. The same policies around academic integrity apply in the project as they do for the rest of CSE 25. In particular:
 - In your work and writing in this course, be honest, specific, and analytical. Honest: Be truthful about your process and observations, including descriptions and citations of resources that are relevant. Specific: Strive to use the terminology and definitions we introduce in class to clearly and precisely describe your observations and conclusions. Analytical: Go deep with your reflections and consider limitations and alternatives to results.
 - Generative artificial intelligence (GenAI) tools may be used in course assignments (not tests or exams) to support your learning as you determine appropriate, as long as you do so honestly through proper documentation, citation, and acknowledgement.
 - When working in a team, take accountability for your contributions to the team and meet your commitments to your teammates.

Project timeline

The project has several milestones to help you stay on track and support your progress. All work will be submitted through Gradescope ((<https://www.gradescope.com/>)).

Team formation

(2% of project grade) Due Week 5 Thursday, April 30, 2026. Deliverable: Gradescope online assignment.

Project proposal

(5% of project grade) Due Week 6 Thursday, May 7, 2026. Deliverable: project proposal (1-2 typed pages) describing project problem statement, dataset, and high-level approach. Upload PDF to Gradescope.

Project mentor meeting(s)

(3% of project grade) Due Week 7 Thursday, May 14, 2026. Deliverable: schedule at least one in-person meeting, during which your group will demo at least three working functions to your mentor TA or tutor. One of the functions needs to load a data set and one of the functions needs to perform some aspect of training. Before this meeting, prepare to discuss your team's next steps and goals for the rest of project, including your time estimates for each stage. You and your mentor will document the meeting in Gradescope online assignment. This mentor meeting will also be a check-in for the effectiveness of collaboration and contributions of the team members.

Project presentation

(10% of project grade) In-person presentations during final exam slot, Tuesday June 9, 2026. Deliverable: 10 minute team presentation and demo of your project.

Project final report

(75% of project grade) Due Tuesday June 9, 2026. Deliverable: final version of project report and code on Gradescope.

Individual contribution component

(5% of project grade) Due Tuesday June 9, 2026. Deliverable: each team member will provide feedback on individual contributions.

Sample projects

Handwritten digit recognition

Problem Statement: Handwritten digit recognition is a classic supervised learning problem and one of the most common entry points into computer vision. Given a small grayscale image of a handwritten digit, the goal is to predict which of the ten digits it shows. In this project, you'll directly compare different machine learning model families on the same data.

You will use the 8x8 handwritten digits dataset bundled with scikit-learn. The dataset is small enough to run on a laptop CPU or on Datahub without a GPU, though larger models may require one. Your team will compare two models on the same train, validation, and test split: a k-nearest neighbors classifier, which predicts a label by taking a majority vote among the closest training examples, and a feed-forward neural network, or FFNN, which learns weights by gradient descent. Both models are implemented from scratch. The FFNN includes a small autograd system of its own, so you can inspect the forward pass, loss computation, and gradient updates without relying on PyTorch or TensorFlow. After comparing accuracy, runtime, and common errors, your team should try several extensions. Examples of this can include scaling up to MNIST, vectorizing the FFNN with NumPy, or changing the network architecture and learning rate. You should try different model families and compare performance between them and within them. For instance, you may vary the size of a given model, experiment with different numbers of epochs and learning rates, etc.

The starter code for this project provides runnable from-scratch implementations of both baselines, along with a shared evaluation harness and comparison table.

Pedestrian Detection in Penn-Fudan

Problem Statement: Object detection is a fundamental challenge in Computer Vision that involves identifying the presence, location, and type of objects within an image. In this project, you will focus specifically on the task of pedestrian detection, a critical component in the development of autonomous vehicles, surveillance systems, and urban safety technology. Unlike simple classification, detection requires the model to not only identify that a person is present but also to provide a precise bounding box around each individual.

For this task, you will use the Penn-Fudan Database for Pedestrian Detection. This dataset contains images of pedestrians in various outdoor scenes, with each pedestrian carefully annotated with a bounding box and a segmentation mask. This is a supervised learning problem where the goal is to train a model that can generalize to realistic, unsegmented scenes and accurately localize multiple people in a single frame. You will implement and train deep learning architectures designed for object localization and evaluate how well they can distinguish pedestrians from complex backgrounds.

The starter code for this project guides you through installing and importing dependencies, downloading the Penn-Fudan dataset, visualizing the data, extracting bounding boxes from the dataset, loading the data into PyTorch, and using PyTorch libraries to create train/validation split.

Image Captioning in MS-COCO

Problem Statement: Automatically describing the content of an image using natural language is a fundamental challenge in AI that bridges the gap between Computer Vision and Natural Language Processing. In this project, you will explore the task of image captioning, where the goal is to generate a coherent English sentence that describes the salient features and actions within a given photograph. This task is significantly more complex than standard image classification; a model must not only recognize individual objects but also understand their relationships, attributes, and the overall context of the scene. Such technology has profound real-world applications, such as assisting visually impaired individuals by converting visual web content into descriptive audio.

You will utilize the Microsoft Common Objects in Context (MS COCO) dataset, a large-scale benchmark containing over 330,000 images. Each image in the training set is paired with at least five independent human-generated captions, providing a rich source of linguistic diversity for supervised learning. Typically this problem is solved with an encoder-decoder architecture. In the starter code, we've provided a functioning Convolution Neural Network (CNN) encoder for you. Your task is to implement various decoders to generate the corresponding text sequence.

The starter code for this project guides you through installing and importing dependencies, downloading the MS-COCO dataset, exploring the existing captions, building a vocabulary of tokens used in captions, creating a custom representation of the dataset, and loading a pre-trained encoder.

Movie recommendation systems in Movielens

Problem Statement: In the modern digital landscape, users are often overwhelmed by a vast amount of available content, making it difficult to discover movies that align with their specific tastes. Recommender systems solve this "information filtering" problem by predicting the "rating" or "preference" a user would give to an item they have not yet seen. In this project, you will build a recommendation engine that generates personalized suggestions, a core technology used by industry leaders like Netflix and Amazon to improve user satisfaction and retention. This task involves addressing classic machine learning challenges such as "data sparsity" (most users only rate a few movies) and the "cold start" problem (how to recommend items to new users with no history).

You will work with the MovieLens dataset, a widely used benchmark in the field of recommender systems that contains millions of movie ratings, titles, and genre metadata. Your team will explore and implement different algorithmic approaches, such as Collaborative Filtering—which finds similarities between users or items based on rating patterns—and Content-Based Filtering, which suggests movies similar in genre or cast to those a user has liked in the past. You may also explore hybrid models or matrix factorization techniques like Singular Value

Decomposition (SVD) to improve accuracy. The goal is to develop a robust system that can effectively capture individual preferences and deliver high-quality, relevant recommendations.

The starter code for this project guides you through installing and importing dependencies, downloading the MovieLens dataset, exploring the existing data, parsing genres (for content-based filtering), and creating a the train/test split in the data set.

Learning to play Blackjack

Problem Statement: Reinforcement Learning (RL) is a unique area of machine learning where an agent learns to make decisions by interacting with an environment to maximize a reward. In this project, you will apply RL to the game of Blackjack, where the goal is to obtain a hand total as close to 21 as possible without exceeding it. Unlike supervised learning, there are no labeled "correct" moves provided; instead, the agent must learn through trial and error—discovering which actions (like "hitting" for another card or "standing" to keep its current hand) lead to a win or a loss.

While there are many methods, one example is to use the OpenAI Gym (Gymnasium) Blackjack-v1 environment. In this simulation, the agent receives a state observation consisting of its current hand sum, the dealer's visible card, and whether it holds a "usable ace". Your team will implement an RL algorithm, such as Q-Learning or Monte Carlo Control, to train the agent over thousands of simulated hands. You will explore the trade-off between exploration (trying new moves) and exploitation (using known winning moves) by implementing strategies like epsilon-greedy. The final goal is to develop a policy that can match or even exceed basic human playing strategies.

Advice from previous students in CSE 25

"I advise future students to read as much material as possible to see what has been done. Research is about working off the contributions of others, so in order to effectively work through a project like this, reading sufficient and relevant material is a must."

"Changing the provided Jupyter Notebook (.ipynb) code into standard Python (.py) will allow more efficient utilization of your hardware."

"To any future students who want to create a model that generates wallpaper art, we recommend sticking with the process and being willing to experiment. There were times when the loss increased, but the paintings continued to improve. There were also times when the trainer would fail due to a "learning rate" factor that was set too high. However, understanding why these things happen, including the fact that just because your trainer fails doesn't mean there is a problem with your code, is important. Secondly, when it comes to generating wallpapers, numerical analysis does not have to be the only judge in quantifying the art. Although that helps track the progress of an image generator, having people judge is important."

"With very little time, try not to get overly ambitious. You can only do so much in such a small time frame so keep the project simple. Also, in terms of organization, creating an account dedicated for this project is helpful so that everyone can access the same Overleaf paper, work on the same Jupyter notebook, and accessing files. We also think it is helpful to use AI for generating code; however, it is important that you know how the code works before using it (because it can perform differently from your expectations)."

"understanding the AI development pipeline, understanding the general idea of how to train a model (finding datasets, train/val/test split, loss minimization, etc.), and knowing what type of model (regression/classification/RL) to use in certain situations [is important]. Knowing these things will help you understand the general progress of the project and still help out with the project despite not having prior experience. Also have a strong understanding of precision (false positive) and recall (false negative) and how that can be used to evaluate the accuracy of a classification model especially on a dataset like the one we used."

"For future students who attempt a similar project, we recommend starting with the simplest or baseline version of the project before going on to implement other mechanics. In the context of this project, this means starting off with understanding how the agent learns from just hit/stand before implementing splitting and doubling down. Additionally, we recommend utilizing artificial intelligence tools as an assistant for research and brainstorming purposes, as long as you understand what it is generating."

Project structure

Project code

You will build code to load and process data, build and train a model to learn something using the data, and evaluate this model. All files that are required to run the code need to be submitted, either via a zip file uploaded to Gradescope or as a link to a shared (e.g. Github) repository. The instructional team should be able to access, run, and reproduce the results reported in your project.

Project report

The project write-up is expected to be prepared as a single report per team, It should be typed (preferably using the general style of NeurIPS conference - a LATEX template is available on Overleaf). The report should have **at most 7 content pages**. References and any appendices do not count toward this limit. If you are using one of the example projects, do not copy the project descriptions above; please write the relevant sections in your own words.

1. Introduction

- Clearly state the problem or question your project addresses. Describe its relevance and importance.
- Discuss previous and related work to your chosen problem.

2. Data Sourcing and Processing

- Describe the dataset being used. Where is it from? What features are being used? What are their dimensions? etc.
- Explain any preprocessing steps, such as handling missing data, discretizations, or feature selection. Why are these processing steps important for your problem?

3. Model Description

- Describe your model architecture i.e. What type of model is it? How many parameters does it have? Describe the layers used. Etc.
- Describe comparison baseline models.

4. Results and Discussion

- Explore and compare multiple configurations. Summarize how these changes affect performance and stability, and explain any observed differences.
- Present quantitative results (e.g., loss, accuracy, training time) in a presentable form (tables, graphs, etc.).
- Discuss qualitative insights: what aspects of the data does the model capture or miss?

5. Conclusion

- Summarize key findings and model performance.
- Discuss limitations and propose potential extensions or improvements.

6. Reflections & Contributions

- Any suggestions or advice for future students trying such a project?
- For each team member, include a brief summary outlining their specific tasks and contributions.
- Each member should also include a brief individual reflection on their personal learning from the project.
- If you've used Gen AI: document where, to what extent, and how GenAI tools were used. Be as specific as possible to describe the decision-making process for using or not using them for specific tasks, and reflect on whether you agree with these choices in retrospect.

7. References

- Cite or reference all external papers, datasets, and tools used in your project

Acknowledgement

The design of the project in this class draws heavily from that in CSE 25 at UC San Diego in Winter 2026, designed and taught by Prof. Trevor Bonjour. Shreyas Kadekodi and Eric Song contributed significantly to the design and code of the sample projects.